Security Assessment Report

Generated: 2026-05-06 00:17
Report ID: 0c275ce0

Security Assessment Report

Target: /workspace/uploads/8dc8a7b2939f

Primary Component Analyzed: server.py

Assessment Basis: Automated SAST results from Bandit, SonarQube, and Safety

1. Executive Summary

The automated assessment of the application codebase identified several code quality and maintainability concerns, with four critical findings reported by SonarQube and two major code smells. Bandit did not identify any securityspecific issues in the scanned Python file, and Safety could not evaluate dependencies because no requirements.txt file was present.

The most significant issues are high cognitive complexity functions in server.py, which increase the likelihood of logic flaws, security bypasses, and maintenance errors during future changes. While these findings are not direct exploitable vulnerabilities by themselves, they represent meaningful engineering risk in a securitysensitive application.

Overall Risk Level: Medium

This rating is based on the absence of confirmed direct code vulnerabilities, combined with the presence of several critical maintainability issues that can contribute to downstream security weaknesses.

2. Risk Overview

Findings by Severity

Critical: 4

High: 0

Medium: 0

Low: 0

Informational: 2

Risk Interpretation

Critical: Complex functions can conceal securityrelevant logic errors and make secure review, testing, and future remediation difficult.

Informational: Commentedout code indicates poor code hygiene and can lead to confusion, accidental reactivation of legacy logic, or misinterpretation during maintenance.

No direct highrisk exploit findings were confirmed by the available automated scan results.

3. Detailed Findings

Critical Vulnerabilities

Finding 1 — Excessive Cognitive Complexity in server.py Function at Line 120

Severity: Critical

CVSS Score (estimated): 8.1

OWASP Category:

A04: Insecure Design

A06: Vulnerable and Outdated Components *(indirectly, due to maintainability risk impacting secure lifecycle practices)*

Affected Component:

server.py — function starting at line 120

Description:

SonarQube flagged this function for excessive cognitive complexity, with a measured score of 26, exceeding the allowed threshold of 15.

Highly complex functions are difficult to understand, test, and safely modify.

In securitysensitive code, this significantly increases the chance of introducing logic flaws, authorization mistakes, inputhandling defects, or inconsistent error handling during future updates.

Impact:

Attackers may benefit indirectly if future modifications introduce exploitable flaws.

Security controls embedded in complex logic may be bypassed or incorrectly enforced.

Business impact includes increased maintenance cost, slower patching, and higher likelihood of regression during security fixes.

Proof of Concept (if possible):

No direct exploit was identified from the scan output.

The risk is latent: security defects may emerge when developers modify or extend the function due to its complexity.

Recommended Remediation:

Refactor the function into smaller, singlepurpose subroutines.

Apply separation of concerns for parsing, validation, decisionmaking, and response generation.

Add unit tests for each logical branch and boundary condition.

Enforce code review standards for complexity thresholds in CI/CD.

Consider static analysis gating to prevent future regressions.

Finding 2 — Excessive Cognitive Complexity in server.py Function at Line 151

Severity: Critical

CVSS Score (estimated): 8.1

OWASP Category:

A04: Insecure Design

Affected Component:

server.py — function starting at line 151

Description:

SonarQube reported this function with a cognitive complexity of 17, exceeding the acceptable limit of 15.

The nested control flow and multiple branching paths make the logic difficult to reason about and verify.

Complex branching in application logic often leads to overlooked edge cases and inconsistent enforcement of securityrelevant checks.

Impact:

Increased probability of logic errors affecting access control, validation, or workflow integrity.

Future maintenance may introduce subtle vulnerabilities.

Operationally, this increases defect density and makes incident response more difficult when issues arise.

Proof of Concept (if possible):

No direct proofofconcept can be derived from the scan results alone.

The concern is structural and preventive rather than immediately exploitable.

Recommended Remediation:

Break the function into modular helper methods.

Flatten nested conditionals where possible using guard clauses.

Introduce explicit validation and decision layers.

Add securityfocused tests for each logical branch.

Review the function with a focus on authorization, validation, and errorhandling correctness.

Finding 3 — Excessive Cognitive Complexity in server.py Function at Line 198

Severity: Critical

CVSS Score (estimated): 8.1

OWASP Category:

A04: Insecure Design

Affected Component:

server.py — function starting at line 198

Description:

SonarQube flagged this function with a cognitive complexity of 25, substantially above the allowed threshold.

The issue indicates a deeply branched code path with multiple nested decision points.

Such functions are especially risky in backend services because they can conceal unsafe assumptions or inconsistent handling of untrusted input.

Impact:

Elevated risk of future security defects during changes.

Increased chance that one execution path behaves differently from others, potentially weakening business logic enforcement.

Can impair auditing and secure review of authentication, orchestration, or requesthandling logic.

Proof of Concept (if possible):

No direct exploit demonstrated in the scan results.

The finding reflects maintainability and correctness risk that can become exploitable after code changes.

Recommended Remediation:

Refactor into smaller, wellnamed units with narrow responsibilities.

Replace deep nesting with early returns and clear validation routines.

Add regression tests covering all branches and exception paths.

Perform a manual security review of all logic handled by this function.

Finding 4 — Excessive Cognitive Complexity in server.py Function at Line 198

Severity: Critical

CVSS Score (estimated): 8.1

OWASP Category:

A04: Insecure Design

Affected Component:

server.py — function starting at line 198

Description:

The SonarQube issue appears as a second complexityrelated finding tied to the same function area, indicating another separate overcomplex branch structure or hotspot in the same logical region.

Cognitive complexity at this level reduces code clarity and increases the likelihood of securityrelevant defects.

In application security, complexity often correlates with poor testability and incomplete validation coverage.

Impact:

Increases likelihood of future vulnerabilities being introduced unnoticed.

Makes secure refactoring more difficult and increases operational risk.

May lead to inconsistent handling of requests or state transitions.

Proof of Concept (if possible):

Not applicable from the available scan data.

Recommended Remediation:

Split the logic into reusable components.

Reduce branching and nesting.

Document assumptions and expected states.

Implement comprehensive tests and code quality gates.

High Vulnerabilities

No highseverity vulnerabilities were reported by the provided automated scan results.

Medium Vulnerabilities

No mediumseverity vulnerabilities were reported by the provided automated scan results.

Low Vulnerabilities

No lowseverity vulnerabilities were reported by the provided automated scan results.

Informational Findings

Finding 5 — CommentedOut Code in server.py at Line 26

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A04: Insecure Design *(indirect maintainability concern)*

Affected Component:

server.py — line 26

Description:

SonarQube flagged commentedout code.

While not a direct security vulnerability, leftover code fragments can create confusion, mislead reviewers, and increase the chance that obsolete logic is mistakenly reintroduced.

In securitysensitive projects, stale code can obscure the true attack surface and complicate audits.

Impact:

Maintenance overhead increases.

Reviewers may misinterpret intended logic.

Legacy code may be accidentally restored or copied into production paths.

Proof of Concept (if possible):

Not applicable. This is a hygiene issue rather than an exploitable defect.

Recommended Remediation:

Remove commentedout code from the repository.

Preserve historical code in version control rather than inline comments.

Use clear commit messages and changelogs to retain context.

Finding 6 — CommentedOut Code in server.py at Line 34

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A04: Insecure Design

Affected Component:

server.py — line 34

Description:

SonarQube identified another instance of commentedout code.

Repeated commented code indicates code hygiene issues and can make the source harder to audit and maintain.

From a security perspective, it may hide deprecated functionality or confuse developers about active behavior.

Impact:

Reduced readability and maintainability.

Higher chance of human error during code modification.

Potential confusion during incident analysis or security review.

Proof of Concept (if possible):

Not applicable.

Recommended Remediation:

Remove obsolete commented code.

If the logic is needed later, recover it from version control.

Enforce repository hygiene standards in code review.

4. Recommendations (Global)

1. Refactor highcomplexity functions

Break down large functions into smaller units with single responsibilities.

Reduce branching and nested conditions.

Add helper methods for validation, decisionmaking, and error handling.

2. Introduce securityfocused testing

Add unit tests and integration tests for all logic branches.

Include negative tests for malformed or unexpected input.

Prioritize coverage for authentication, authorization, and state transitions.

3. Strengthen secure coding practices

Validate all external input before use.

Use explicit error handling and failsafe defaults.

Avoid deeply nested conditionals in requestprocessing logic.

4. Improve dependency governance

Add a requirements.txt or equivalent dependency manifest.

Scan dependencies regularly using Safety, pipaudit, or similar tooling.

Keep dependencies pinned and updated to known secure versions.

5. Enforce code quality gates

Set maximum thresholds for cognitive complexity and code smells in CI/CD.

Block merges when critical maintainability/security indicators are exceeded.

Require peer review for sensitive application logic.

6. Maintain repository hygiene

Remove commentedout code and dead code.

Keep source files concise and readable.

Document intended behavior in code comments or external documentation, not obsolete snippets.

5. Conclusion

The assessment did not identify direct exploitable vulnerabilities through Bandit, but it did reveal multiple critical codequality risks in server.py, primarily related to excessive cognitive complexity. These findings do not confirm an active exploit path; however, they materially increase the likelihood of future security defects and make the application harder to maintain securely.

Immediate remediation should focus on refactoring the identified complex functions, removing commentedout code, and establishing dependency and quality controls. While the current confirmed risk is Medium overall, the critical findings warrant prompt attention to prevent the introduction of more severe issues in future development cycles.